```
from google.colab import drive
drive.mount('/content/drive/')
```

```
Drive already mounted at /content/drive/; to attempt to forcibly remount, call drive.mount("/content/drive/", force_remount=True).
```

```
!pip install spectral
```

```
Looking in indexes: https://pypi.org/simple, https://us-python.pkg.dev/colab-wheels/public/simple/
Requirement already satisfied: spectral in /usr/local/lib/python3.9/dist-packages (0.23.1)
Requirement already satisfied: numpy in /usr/local/lib/python3.9/dist-packages (from spectral) (1.22.4)
```

```
%tensorflow_version 2.x
import tensorflow as tf
import random
from tensorflow.keras import Sequential,layers
from tensorflow.keras.models import Model
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import ModelCheckpoint
import statistics
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report, cohen_kappa_score
import tensorflow_probability as tfp
from operator import truediv
from tensorflow.compat.v1.distributions import Bernoulli
from plotly.offline import init_notebook_mode

import numpy as np
import matplotlib.pyplot as plt
import scipy.io as sio
import os
import spectral
import tensorflow as tf
from tensorflow.python.keras import backend as K

init_notebook_mode(connected=True)
%matplotlib inline
```

```
Colab only includes TensorFlow 2.x; %tensorflow_version has no effect.
```

```
# Global variables
im_width, im_height, im_depth, im_channel = 11,11,30, 1
mc_loss_weight = 5
```

Model Construction

```
def calc_euclidian_dists(x, y):
  # x : (n,d)
  # y : (m,d)
    n = x.shape[0]
    m = y.shape[0]
    x = tf.tile(tf.expand_dims(x, 1), [1, m, 1])
    y = tf.tile(tf.expand_dims(y, 0), [n, 1, 1])
    return tf.reduce_mean(tf.math.pow(x - y, 2), 2)
```

```
def _bernoulli(shape, mean):
    return tf.nn.relu(tf.sign(mean - tf.random.uniform(shape, minval=0, maxval=1, dtype=tf.float32)))
```

```
class DropBlock3D(tf.keras.layers.Layer):
    def __init__(self, keep_prob, block_size, scale=True, **kwargs):
        super(DropBlock3D, self).__init__(**kwargs)
        self.keep_prob = float(keep_prob) if isinstance(keep_prob, int) else keep_prob
        self.block_size = int(block_size)
        self.scale = tf.constant(scale, dtype=tf.bool) if isinstance(scale, bool) else scale

    def compute_output_shape(self, input_shape):
        return input_shape

    def build(self, input_shape):
        assert len(input_shape) == 5
        _, self.d, self.h, self.w, self.channel = input_shape.as_list()
        # pad the mask
        p0 = (self.block_size - 1) // 2
        p1 = (self.block_size - 1) - p0
        self.padding = [[0, 0], [p0, p1], [p0, p1], [p0, p1], [0, 0]]
```

```
        self.set_keep_prob()
        super(DropBlock3D, self).build(input_shape)

    def call(self, inputs, training=True, **kwargs):
        def drop():
            mask = self._create_mask(tf.shape(inputs))
            output = inputs * mask
            output = tf.cond(self.scale,
                             true_fn=lambda: output * tf.compat.v1.to_float(tf.size(mask)) / tf.reduce_sum(mask),
                             false_fn=lambda: output)
            return output

        if training is None:
            training = K.learning_phase()
        output = tf.cond(tf.logical_or(tf.logical_not(training), tf.equal(self.keep_prob, 1.0)),
                         true_fn=lambda: inputs,
                         false_fn=drop)
        return output

    def set_keep_prob(self, keep_prob=None):
        """This method only supports Eager Execution"""
        if keep_prob is not None:
            self.keep_prob = keep_prob
        d, w, h = tf.compat.v1.to_float(self.d), tf.compat.v1.to_float(self.w), tf.compat.v1.to_float(self.h)
        self.gamma = ((1. - self.keep_prob) * (d * w * h) / (self.block_size ** 3) /
                      ((d - self.block_size + 1) * (w - self.block_size + 1) * (h - self.block_size + 1)))

    def _create_mask(self, input_shape):
        sampling_mask_shape = tf.stack([input_shape[0],
                                        self.d - self.block_size + 1,
                                        self.h - self.block_size + 1,
                                        self.w - self.block_size + 1,
                                        self.channel])
        mask = _bernoulli(sampling_mask_shape, self.gamma)
        mask = tf.pad(mask, self.padding)
        mask = tf.nn.max_pool3d(mask, [1, self.block_size, self.block_size, self.block_size, 1], [1, 1, 1, 1, 1], 'SAME')
        mask = 1 - mask
        return mask
```

```
input_layer = layers.Input(shape = (im_height, im_width, im_depth, im_channel))
out1 = layers.Conv3D(filters=8, kernel_size=(3,3,7), activation='relu',input_shape=(im_height, im_width, im_depth, im_channel),padding='s
out1 = DropBlock3D(0.7,3)(out1)
out2 = layers.Conv3D(filters=16, kernel_size=(3,3,5), activation='relu',padding='same')(out1)
out2 = DropBlock3D(0.7,3)(out2)
out3 = layers.Conv3D(filters=32, kernel_size=(3,3,3), activation= 'relu')(out2)
out3 = layers.Reshape((out3.shape[1], out3.shape[2], out3.shape[3]*out3.shape[4]))(out3)
out3 = layers.Conv2D(filters=64, kernel_size=(3,3), activation='relu')(out3)
out4 = layers.Flatten()(out3)
out4 = layers.Dropout(0.4)(out4, training=True)
out4 = layers.Dense(256, activation='relu')(out4)
out5 = layers.Dropout(0.4)(out4,training=True)
out5 = layers.Dense(128, activation='relu')(out5)
#out5 = layers.Dropout(0.4)(out5, training=True)
#out6 = layers.Dense(16, activation='relu')(out5)
model = Model(inputs=input_layer,outputs=out5)
model.summary()
```

```
Model: "model_1"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 input_2 (InputLayer)        [(None, 11, 11, 30, 1)]   0

 conv3d_3 (Conv3D)           (None, 11, 11, 30, 8)     512

 drop_block3d_2 (DropBlock3D  (None, 11, 11, 30, 8)    0
 )

 conv3d_4 (Conv3D)           (None, 11, 11, 30, 16)    5776

 drop_block3d_3 (DropBlock3D  (None, 11, 11, 30, 16)   0
 )

 conv3d_5 (Conv3D)           (None, 9, 9, 28, 32)      13856

 reshape_1 (Reshape)         (None, 9, 9, 896)         0

 conv2d_1 (Conv2D)           (None, 7, 7, 64)          516160

 flatten_1 (Flatten)         (None, 3136)              0

 dropout_2 (Dropout)         (None, 3136)              0

 dense_2 (Dense)             (None, 256)               803072
```

```
 dropout_3 (Dropout)         (None, 256)               0

 dense_3 (Dense)             (None, 128)               32896

=================================================================
Total params: 1,372,272
Trainable params: 1,372,272
Non-trainable params: 0
_________________________________________________________________
```

```
class Prototypical(Model):
    def __init__(self, model, w, h, d, c):
        super(Prototypical, self).__init__()
        self.w, self.h, self.d, self.c = w, h, d, c
        self.encoder = model

    def call(self, support, query, support_labels, query_labels, K, C, N,n_times,training=True):
      n_class = C                                                          #10
      n_support = K                                                        #5
      n_query = N                                                          #15
      Temperature = 1.8                                                    # Temperature = 1.1 to 3.0

      if training == True :
        loss = 0
        mc_predictions = []                                                # list of predictions for multiple passes
        for j in range(n_times) :
          y = np.zeros((int(C*N),C))                                         #(150,10)
          for i in range(int(C*N)) :
            x = support_labels.index(query_labels[i])                      # creation of 1-hot for true labels
            y[i][x] = 1.                                             # n_times passing every query image for calculating variance
          cat = tf.concat([support,query], axis=0)                         # [200,9,9,20,1])
          z = self.encoder(cat)                                            # [200, 320]
          # Divide embedding into support and query
          z_prototypes = tf.reshape(z[:n_class * n_support],[n_class, n_support, z.shape[-1]])   #[10, 5, 320])
          # Prototypes are means of n_support examples
          z_prototypes = tf.math.reduce_mean(z_prototypes, axis=1)         #[10, 320]
          z_query = z[n_class * n_support:]                                #[150, 320]
          # Calculate distances between query and prototypes
          dists = calc_euclidian_dists(z_query, z_prototypes)              #[150, 10]
          # log softmax of calculated distances
          log_p_y = tf.nn.log_softmax(-dists, axis=-1)                     #[150, 10]
          loss1 = -tf.reduce_mean((tf.reduce_sum(tf.multiply(y, log_p_y), axis=-1)))   #loss for the current pass
          loss += loss1                                                    # adding loss for each pass
          softmax_predictions = tf.nn.softmax(-dists, axis=-1)             # prediction probability for the search-space classes per
          new_distance = dists/Temperature                                 # T = 1.1 to 3.0 for Temperature Scaling
          predictions = tf.nn.softmax(-new_distance, axis=-1)              # prediction probability with temperature scaling
          # print('Loop Number :', j)
          # print("A" * 20)
          # print("Dists : ", dists[0])
          # print("B" * 20)
          # print('New Distance (T applied): ', new_distance[0])
          # print("C" * 20)
          # print("Softmax Predictions : ", softmax_predictions[0])
          # print("D" * 20)
          # print("Temperature Scaling Predictions : ", predictions[0])
          # print()
          mc_predictions.append(predictions)
        y = np.zeros((int(C*N),C))
        for i in range(int(C*N)) :
            x = support_labels.index(query_labels[i])                      # creation of 1-hot for true labels
            y[i][x] = 1.
        mc_predictions = tf.convert_to_tensor(np.reshape(np.asarray(mc_predictions),(n_times,int(C*N),C)))  #(n_times,150,10)
        std_predictions = tf.math.reduce_std(mc_predictions,axis=0)
        std = tf.reduce_sum(tf.reduce_sum(tf.multiply(std_predictions,y),axis=1))
        # print('std', std)
        loss += mc_loss_weight*std
        temp_list.append(std)
        # calculating mean accuracy
        mean_predictions = tf.reduce_mean(mc_predictions,axis=0)         # mean prediction probability for each class (150,10)
        mean_eq = tf.cast(tf.equal(                                      # accuracy for the current pass
            tf.cast(tf.argmax(mean_predictions, axis=-1), tf.int32),
            tf.cast(tf.argmax(y,axis=-1), tf.int32)), tf.float32)
        mean_accuracy = tf.reduce_mean(mean_eq)
        mean_predictions = tf.reduce_mean(mc_predictions,axis=0)         # mean prediction probability for each class (150,10)
        return loss, mean_accuracy, mean_predictions
      if training == False :
        loss = 0
        mc_predictions = []                                                # list of predictions for multiple passes
        for j in range(n_times) :                                          # n_times passing the query images for variance calculati
          y = np.zeros((int(C*N),C))                                       # (150,10)
          for i in range(int(C*N)) :
```

```
        x = support_labels.index(query_labels[i])                          # creation of 1-hot for the true labels
        y[i][x] = 1.
      # merge support and query to forward through encoder
      cat = tf.concat([support,query], axis=0)                             # [200,9,9,20,1]
      z = self.encoder(cat)                                                # [200, 320]
      # Divide embedding into support and query
      z_prototypes = tf.reshape(z[:n_class * n_support],[n_class, n_support, z.shape[-1]])   #[10, 5, 320])
      # Prototypes are means of n_support examples
      z_prototypes = tf.math.reduce_mean(z_prototypes, axis=1)             #[10, 320]
      z_query = z[n_class * n_support:]                                    #[150, 320]
      # Calculate distances between query and prototypes
      dists = calc_euclidian_dists(z_query, z_prototypes)                  #[150, 10]
      # log softmax of calculated distances
      log_p_y = tf.nn.log_softmax(-dists, axis=-1)                         #[150, 10]
      loss1 = -tf.reduce_mean((tf.reduce_sum(tf.multiply(y, log_p_y), axis=-1)))
      loss += loss1
      softmax_predictions = tf.nn.softmax(-dists, axis=-1)                 # prediction probability for the search-space classes per
      new_distance = dists/Temperature                                             # T = 1.1 to 3.0 for Temperature Scaling
      predictions = tf.nn.softmax(-new_distance, axis=-1)                  # prediction probability with temperature scaling
      # print('Loop Number : ', j)
      # print("A" * 20)
      # print("Dists : ", dists[0])
      # print("B" * 20)
      # print('New Distance (T applied): ', new_distance[0])
      # print("C" * 20)
      # print("Softmax Predictions : ", softmax_predictions[0])
      # print("D" * 20)
      # print("Temperature Scaling Predictions : ", predictions[0])
      # print()
      mc_predictions.append(predictions)
    y = np.zeros((int(C*N),C))                                           # (150,10)
    for i in range(int(C*N)) :
        x = support_labels.index(query_labels[i])                          # creation of 1-hot for the true labels
        y[i][x] = 1.
    mean_predictions = tf.reduce_mean(mc_predictions,axis=0)               # mean prediction probability for each class (150,10)
    mean_eq = tf.cast(tf.equal(                                            # accuracy for the current pass
        tf.cast(tf.argmax(mean_predictions, axis=-1), tf.int32),
        tf.cast(tf.argmax(y,axis=-1), tf.int32)), tf.float32)
    mean_accuracy = tf.reduce_mean(mean_eq)
    mean_pred_index = tf.argmax(mean_predictions,axis=1)
    # mean class-wise accuracies
    mean_correct_class = [[] for i in range(tC)]
    mean_correct_pred = [[] for i in range(tC)]
    classwise_mean_acc = [[] for i in range(tC)]
    for i in range(int(C*N)):
      x = support_labels.index(query_labels[i])
      mean_correct_class[x].append('4')
      if(mean_pred_index[i] == x) :
        mean_correct_pred[x].append('4')
    for i in range(tC) :
       z = len(mean_correct_pred[i])/len(mean_correct_class[i])
       classwise_mean_acc[i].append(z)
    #std calculation
    std = 0
    for i in range(int(C*N)) :
       x = support_labels.index(query_labels[i])
       p_i = np.array([p[i,:] for p in mc_predictions])
       std_i = tf.math.reduce_std(p_i,axis=0)
       std_i_true = std_i[x]
       std += std_i_true                                                   # adding std of each class
    # print('std',std)
    loss += mc_loss_weight*std
    y = np.zeros((int(C*N),C))                                           # (150,10)
    for i in range(int(C*N)) :
        x = support_labels.index(query_labels[i])                          # creation of 1-hot for the true labels
        y[i][x] = 1.
    return loss, mc_predictions, mean_accuracy, classwise_mean_acc, y


  def save(self, model_path):
    self.encoder.save(model_path)

  def load(self, model_path):
    self.encoder(tf.zeros([1, self.w, self.h, self.c]))
    self.encoder.load_weights(model_path)
```

Data Loading

[ ] ↳ *10 cells hidden*

Training

```
ProtoModel = Prototypical(model,im_width, im_height, im_depth, im_channel)
optimizer = tf.keras.optimizers.Adam(0.001)          #Adam(0.001)
n_times = 5
#std_list = []
```

```
# Metrics to gather
train_loss = tf.metrics.Mean(name='train_loss')
train_acc = tf.metrics.Mean(name='train_accuracy')

def train_step(support, query, support_labels, query_labels, K, C, N):
    # Forward & update gradients
    with tf.GradientTape() as tape:
        loss, mean_accuracy, mean_predictions = ProtoModel(support, query, support_labels, query_labels, K, C, N,n_times,training=True)
    gradients = tape.gradient(loss, model.trainable_variables)
    optimizer.apply_gradients(zip(gradients, model.trainable_variables))

    # Log loss and accuracy for step
    train_loss(loss)
    train_acc(mean_accuracy)
```

```
checkpoint_dir = '/content/drive/My Drive/Research/SAMPLE_CODE_TF2_Keras/Inprogress/Hyperspectral_Classification/Training checkpoints/IP/
checkpoint_prefix = os.path.join(checkpoint_dir, "ckpt")
checkpoint = tf.train.Checkpoint(optimizer=optimizer,
                                 ProtoModel = ProtoModel)
```

```
n_episodes = 100
temp_list = []
for epoch in range(50): # n_epochs-140
    train_loss.reset_states()
    train_acc.reset_states()

    for epi in range(n_episodes):
        tquery_patches, tsupport_patches, query_labels, support_labels = new_episode(patches_class_ip,K1,C,N,train_class_labels)
        train_step(tsupport_patches, tquery_patches,support_labels, query_labels, K1, C, N)
        template = 'Epoch {}, Episode {}, Train Loss: {:.2f}, Train Accuracy: {:.2f}'
        print(template.format(epoch+1, epi+1,train_loss.result(),train_acc.result()*100))
    if epoch % 5 == 0 and epoch != 0 :
        checkpoint.save(file_prefix = checkpoint_prefix)
```

```
Epoch 50, Episode 83, Train Loss: 1.09, Train Accuracy: 99.98
Epoch 50, Episode 84, Train Loss: 1.08, Train Accuracy: 99.98
Epoch 50, Episode 85, Train Loss: 1.07, Train Accuracy: 99.98
Epoch 50, Episode 86, Train Loss: 1.06, Train Accuracy: 99.98
Epoch 50, Episode 87, Train Loss: 1.06, Train Accuracy: 99.98
Epoch 50, Episode 88, Train Loss: 1.12, Train Accuracy: 99.98
Epoch 50, Episode 89, Train Loss: 1.12, Train Accuracy: 99.99
Epoch 50, Episode 90, Train Loss: 1.11, Train Accuracy: 99.99
Epoch 50, Episode 91, Train Loss: 1.11, Train Accuracy: 99.99
Epoch 50, Episode 92, Train Loss: 1.11, Train Accuracy: 99.99
Epoch 50, Episode 93, Train Loss: 1.10, Train Accuracy: 99.99
Epoch 50, Episode 94, Train Loss: 1.11, Train Accuracy: 99.99
Epoch 50, Episode 95, Train Loss: 1.11, Train Accuracy: 99.99
Epoch 50, Episode 96, Train Loss: 1.12, Train Accuracy: 99.99
Epoch 50, Episode 97, Train Loss: 1.11, Train Accuracy: 99.99
Epoch 50, Episode 98, Train Loss: 1.12, Train Accuracy: 99.99
Epoch 50, Episode 99, Train Loss: 1.14, Train Accuracy: 99.99
Epoch 50, Episode 100, Train Loss: 1.13, Train Accuracy: 99.99
```

Tuning_5

```
tune_set_5 = [[] for i in range(6)]
for j in range(6) :
  tune_set_5[j] = test_patches_class[j][:5,:,:,:,:]   # for each class first 5 samples taken
std_tune_5 = []
```

```
def tune_episode(tune_set,tC,tK,tN,test_class_labels) :
  selected_classes = np.random.choice(test_class_labels,tC,replace=False)
  support_labels  = list(selected_classes)
  query_labels = []
  support_patches = []
  query_patches = []
  for x in selected_classes :
    y = test_class_labels.index(x)
    np.random.shuffle(tune_set[y])
    support_imgs = tune_set[y][:tK,:,:,:,:]    #Support 1, Query 4
    query_imgs = tune_set[y][tK:5,:,:,:,:]
    support_patches.extend(support_imgs)
    query_patches.extend(query_imgs)
    for i in range(tN) :
      query_labels.append(x)
  temp1 = list(zip(query_patches, query_labels))
  random.shuffle(temp1)
  query_patches, query_labels = zip(*temp1)
  query_patches = tf.convert_to_tensor(np.reshape(np.asarray(query_patches),(tC*tN,im_height,im_width,im_depth,1)),dtype=tf.float32)
  support_patches = tf.convert_to_tensor(np.reshape(np.asarray(support_patches),(tC*tK,im_height,im_width,im_depth,1)),dtype=tf.float32)
  return query_patches, support_patches, query_labels, support_labels
```

```
query_patches, support_patches, query_labels, support_labels = tune_episode(tune_set_5,4,1,4,test_class_labels)
```

```
# Metrics to gather
tune_loss = tf.metrics.Mean(name='tune_loss')
tune_acc = tf.metrics.Mean(name='tune_accuracy')

def tune_step(support, query, support_labels, query_labels, K, C, N):
    # Forward & update gradients
    with tf.GradientTape() as tape:
        loss, mean_accuracy, mean_predictions = ProtoModel(support, query, support_labels, query_labels, K, C, N,n_times,training=True)
    gradients = tape.gradient(loss, model.trainable_variables)
    optimizer.apply_gradients(zip(gradients, model.trainable_variables))

    # Log loss and accuracy for step
    tune_loss(loss)
    tune_acc(mean_accuracy)
```

```
checkpoint_dir1 = '/content/drive/My Drive/Research/SAMPLE_CODE_TF2_Keras/Inprogress/Hyperspectral_Classification/Training checkpoints/IP
checkpoint_prefix1 = os.path.join(checkpoint_dir1, "ckpt")
checkpoint = tf.train.Checkpoint(optimizer=optimizer,
                                 ProtoModel = ProtoModel)
```

```
n_episodes = 100
temp_list = []
std_tune_5 = []
for epoch in range(41):
    tune_loss.reset_states()
    tune_acc.reset_states()
    for epi in range(n_episodes+1):
        tquery_patches, tsupport_patches, query_labels, support_labels = tune_episode(tune_set_5,3,1,4,test_class_labels)
        tune_step(tsupport_patches, tquery_patches,support_labels, query_labels, 1, 3, 4)
```

```
    template = 'Epoch {}, Tune Loss: {:.2f}, Tune Accuracy: {:.2f}'
    print(template.format(epoch+1,tune_loss.result(),tune_acc.result()*100))
    if (epoch+1)%5 == 0 :
      checkpoint.save(file_prefix = checkpoint_prefix1)
```

```
Epoch 1, Tune Loss: 5.03, Tune Accuracy: 99.50
Epoch 2, Tune Loss: 1.17, Tune Accuracy: 100.00
Epoch 3, Tune Loss: 0.15, Tune Accuracy: 100.00
Epoch 4, Tune Loss: 0.11, Tune Accuracy: 100.00
Epoch 5, Tune Loss: 0.28, Tune Accuracy: 100.00
Epoch 6, Tune Loss: 0.05, Tune Accuracy: 100.00
Epoch 7, Tune Loss: 0.08, Tune Accuracy: 100.00
Epoch 8, Tune Loss: 0.04, Tune Accuracy: 100.00
Epoch 9, Tune Loss: 0.04, Tune Accuracy: 100.00
Epoch 10, Tune Loss: 0.01, Tune Accuracy: 100.00
Epoch 11, Tune Loss: 0.08, Tune Accuracy: 100.00
Epoch 12, Tune Loss: 0.13, Tune Accuracy: 100.00
Epoch 13, Tune Loss: 0.13, Tune Accuracy: 100.00
Epoch 14, Tune Loss: 0.08, Tune Accuracy: 100.00
Epoch 15, Tune Loss: 0.04, Tune Accuracy: 100.00
Epoch 16, Tune Loss: 0.18, Tune Accuracy: 100.00
Epoch 17, Tune Loss: 0.02, Tune Accuracy: 100.00
Epoch 18, Tune Loss: 0.30, Tune Accuracy: 100.00
Epoch 19, Tune Loss: 0.14, Tune Accuracy: 100.00
Epoch 20, Tune Loss: 0.00, Tune Accuracy: 100.00
Epoch 21, Tune Loss: 0.03, Tune Accuracy: 100.00
Epoch 22, Tune Loss: 0.03, Tune Accuracy: 100.00
Epoch 23, Tune Loss: 0.02, Tune Accuracy: 100.00
Epoch 24, Tune Loss: 0.01, Tune Accuracy: 100.00
Epoch 25, Tune Loss: 0.03, Tune Accuracy: 100.00
Epoch 26, Tune Loss: 4.75, Tune Accuracy: 100.00
Epoch 27, Tune Loss: 7.07, Tune Accuracy: 100.00
Epoch 28, Tune Loss: 0.50, Tune Accuracy: 100.00
Epoch 29, Tune Loss: 0.36, Tune Accuracy: 100.00
Epoch 30, Tune Loss: 0.51, Tune Accuracy: 100.00
Epoch 31, Tune Loss: 0.36, Tune Accuracy: 100.00
Epoch 32, Tune Loss: 0.36, Tune Accuracy: 100.00
Epoch 33, Tune Loss: 0.00, Tune Accuracy: 100.00
Epoch 34, Tune Loss: 0.03, Tune Accuracy: 100.00
Epoch 35, Tune Loss: 0.03, Tune Accuracy: 100.00
Epoch 36, Tune Loss: 0.00, Tune Accuracy: 100.00
Epoch 37, Tune Loss: 0.00, Tune Accuracy: 100.00
Epoch 38, Tune Loss: 0.08, Tune Accuracy: 100.00
Epoch 39, Tune Loss: 0.01, Tune Accuracy: 100.00
Epoch 40, Tune Loss: 0.00, Tune Accuracy: 100.00
Epoch 41, Tune Loss: 0.13, Tune Accuracy: 100.00
```

Testing

```
def test_episode(test_patches_class,test_class_labels,test_C,test_K,i,f) :
  selected_classes = test_class_labels[i:f]   # [1, 2, 3, 4, 5, 6, 7, 8]
  support_labels = list(selected_classes)
  query_labels = []
  support_patches = []
  query_patches = []
  for x in selected_classes :
    y = test_class_labels.index(x)
    support_imgs = test_patches_class[y][:test_K,:,:,:,:]
    query_imgs = test_patches_class[y][test_K:,:,:,:,:]
    support_patches.extend(support_imgs)
    query_patches.extend(query_imgs)
    for i in range(query_imgs.shape[0]) :
      query_labels.append(x)
  temp1 = list(zip(query_patches, query_labels))
  random.shuffle(temp1)
  query_patches, query_labels = zip(*temp1)
  x = len(query_labels)
  query_patches = tf.convert_to_tensor(np.reshape(np.asarray(query_patches),(x,im_height,im_width,im_depth,1)),dtype=tf.float32)
  support_patches = tf.convert_to_tensor(np.reshape(np.asarray(support_patches),(test_C*test_K,im_height,im_width,im_depth,1)),dtype=tf.f
  return query_patches, support_patches, query_labels, support_labels,x
```

```
query_patches, support_patches, query_labels, support_labels,x = test_episode(test_patches_class,test_class_labels,3,5,0,3)
```

```
# Metrics to gather
test_loss = tf.metrics.Mean(name='test_loss')
test_acc = tf.metrics.Mean(name='test_accuracy')
def test_step(support, query, support_labels, query_labels, K, C, y):
    loss, mc_predictions, mean_accuracy, classwise_mean_acc, y = ProtoModel(support, query, support_labels, query_labels, K, C, y,n_times
    return loss, mc_predictions, mean_accuracy, classwise_mean_acc, y
```

```
for epoch in range(1): #1000
    test_loss.reset_states()
    test_acc.reset_states()
    tquery_patches1, tsupport_patches1, query_labels1, support_labels1, x1 = test_episode(test_patches_class,test_class_labels,3,5,0,3)
    loss1, mc_predictions1, mean_accuracy1, classwise_mean_acc1, y1 = test_step(tsupport_patches1, tquery_patches1,support_labels1, query
    print('OA1',mean_accuracy1)
# Class-wise Accuracy
    for i in range(tC) :
      print('class',i+1,classwise_mean_acc1[i])
    print('loss',loss1)
    tquery_patches2, tsupport_patches2, query_labels2, support_labels2, x2 = test_episode(test_patches_class,test_class_labels,3,5,3,6)
    loss2, mc_predictions2, mean_accuracy2, classwise_mean_acc2, y2 = test_step(tsupport_patches2, tquery_patches2,support_labels2, query
    print('OA2',mean_accuracy2)
# Class-wise Accuracy
    for i in range(tC) :
      print('class',i+4,classwise_mean_acc2[i])
    print('loss',loss2)
```

```
OA1 tf.Tensor(0.5675676, shape=(), dtype=float32)
class 1 [0.6097560975609756]
class 2 [0.5172413793103449]
class 3 [1.0]
loss tf.Tensor(1558.9471, shape=(), dtype=float32)
OA2 tf.Tensor(0.9570957, shape=(), dtype=float32)
class 4 [1.0]
class 5 [0.935]
class 6 [1.0]
loss tf.Tensor(123.79727, shape=(), dtype=float32)
```

Overall Accuracy

```
mean_predictions1 =  tf.reduce_mean(mc_predictions1,axis=0)
mean_predictions2 =  tf.reduce_mean(mc_predictions2,axis=0)
overall_predictions = tf.concat([mean_predictions1,mean_predictions2],axis=0)
overall_true_labels = tf.concat([y1,y2],axis=0)
correct_pred = tf.cast(tf.equal(                                          # accuracy for the current pass
            tf.cast(tf.argmax(overall_predictions, axis=-1), tf.int32),
            tf.cast(tf.argmax(overall_true_labels,axis=-1), tf.int32)), tf.float32)
o_acc = tf.reduce_mean(correct_pred)
print("Overall accuracy:",o_acc)
```

```
Overall accuracy: tf.Tensor(0.76460767, shape=(), dtype=float32)
```

Confusion Matrix

```
from sklearn.metrics import confusion_matrix
from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report
mean_predictions1 =  tf.reduce_mean(mc_predictions1,axis=0)
cm_pred1 = tf.argmax(mean_predictions1, axis=-1)
mean_predictions2 =  tf.reduce_mean(mc_predictions2,axis=0)
cm_pred2 = tf.argmax(mean_predictions2, axis=-1) + 3
overall_predictions = tf.concat([cm_pred1,cm_pred2],axis=0)
cm_true1 = tf.argmax(y1,axis=-1)
cm_true2 = tf.argmax(y2,axis=-1) + 3
overall_true_labels = tf.concat([cm_true1,cm_true2],axis=0)
results = confusion_matrix(overall_true_labels,overall_predictions)
print ('Confusion Matrix :')
print(results)
print ('Report : ')
print (classification_report(overall_true_labels, overall_predictions))
```

```
Confusion Matrix :
[[ 25   0  16   0   0   0]
 [ 84 120  28   0   0   0]
 [  0   0  23   0   0   0]
 [  0   0   0  15   0   0]
 [  0   0   0   3 187  10]
 [  0   0   0   0   0  88]]
Report :
              precision    recall  f1-score   support

           0       0.23      0.61      0.33        41
           1       1.00      0.52      0.68       232
           2       0.34      1.00      0.51        23
           3       0.83      1.00      0.91        15
           4       1.00      0.94      0.97       200
           5       0.90      1.00      0.95        88

    accuracy                           0.76       599
   macro avg       0.72      0.84      0.72       599
```

```
weighted avg       0.90      0.76      0.79       599
```

Kappa Accuracy

```
import sklearn
sklearn.metrics.cohen_kappa_score(overall_true_labels, overall_predictions)
```

```
0.6969069501214755
```

✓ 0s completed at 4:45 PM